Runbook: Bash (Linux, macOS, WSL, CI)

Every command to build the Health-style lab Genesys Cloud lab from a Bash shell. One action per step. After each step there is **You should see**.

If you are on Windows, use `RUNBOOK-POWERSHELL.md` unless you have WSL (Windows Subsystem for Linux). Git Bash alone is not enough: it lacks `jq`.

Before you start: read Part 1 and 2 of `BUILD-MANUAL.md` (a lab org and an OAuth client). **Never run this against a customer's org.**

Step 1. Install the tools (once)

Ubuntu, Debian, or WSL:

```
sudo apt-get update
sudo apt-get install -y git curl jq zip unzip
# Node.js 22: use your preferred method, for example NodeSource or nvm.
# OpenTofu: https://opentofu.org/docs/intro/install/  (or HashiCorp Terraform)
```

macOS (Homebrew):

```
brew install git jq node opentofu
```

Check:

```
git --version; curl --version | head -1; jq --version; node --version; tofu version
```

You should see a version for each. Node should be 22 or newer.

> The scripts use OpenTofu if it is installed, otherwise Terraform. Either works.

Step 2. Make the scripts runnable (once)

```
cd ~/genesys-health-style-lab     # change to where the repo is
chmod +x scripts/bash/*.sh
```

You should see no output.

Step 3. Set your credentials

Paste your own values. **They exist only in this shell session.**

```
export GENESYSCLOUD_OAUTHCLIENT_ID="paste-client-id-here"
export GENESYSCLOUD_OAUTHCLIENT_SECRET="paste-secret-here"
export GENESYSCLOUD_REGION="ap-southeast-2"
```

> Typing a secret in a terminal can store it in your shell history. To avoid that, put a space before the command (in many shells that skips history), or run `history -d` afterwards, or read it silently:

```
        read -rs -p "Secret: " GENESYSCLOUD_OAUTHCLIENT_SECRET; export GENESYSCLOUD_OAUTHC
LIENT_SECRET
```

Do not save these in a file in the repo. Do not paste them into chat.

Step 4. Preflight (changes nothing)

```
        ./scripts/bash/00-preflight.sh
```

You should see: `OK curl found`, `OK jq found`, `OK node found`, `OK tofu found`, `OK Token received for region ap-southeast-2`, and `OK Connected to Genesys org: <your org name>`.

Check the org name is your lab org. If it is not, close the shell.

If you see	Do this
`Set GENESYSCLOUD_OAUTHCLIENT_ID`	Step 3 was not done in this shell
`Could not get a token`	Wrong client ID, secret or region
`'jq' is not installed`	Step 1

Step 5. Package the function

```
        ./scripts/bash/01-package-function.sh
```

You should see: `# tests 8`, `# pass 8`, `# fail 0`, `OK Built .../normalise-reference.zip`.

Step 6. Initialise and validate

```
        ./scripts/bash/02-init.sh
```

You should see: `OK Formatting is clean`, `Success! The configuration is valid.`, `OK Configuration is valid`.

If formatting fails: `cd terraform && tofu fmt -recursive && cd ..`, then run Step 6 again.

After this a file `terraform/.terraform.lock.hcl` exists. **Keep it and commit it.**

Step 6b. The offline plan test (needs no org)

This plans the whole design four ways (minimal, full, surge switch on, sign-in hardening on) with the Genesys provider **mocked**. It needs no credentials and changes nothing. It catches mistakes that `validate` cannot see, such as a queue with too few rings or a percentage given as 70 instead of 0.70.

```
        ( cd terraform && tofu init -backend=false && tofu test )
```

You should see `Success! 4 passed, 0 failed.` OpenTofu 1.8 or Terraform 1.7 or newer is needed.

> It proves the design expands into objects the provider will accept. It does **not** prove Genesys will accept them. Only a real `apply` proves that.

Step 7. Plan (changes nothing)

Smaller build (no AI, quality or web chat), a good first try:

```
./scripts/bash/03-plan.sh minimal
```

Full build:

```
./scripts/bash/03-plan.sh full
```

Full build plus sign-in hardening (**lab org only**; read `terraform/modules/security/main.tf` first, because a wrong setting can lock everyone out, including you, so keep a second admin session open):

```
./scripts/bash/03-plan.sh hardened
```

You should see a long list and a line like `Plan: N to add, 0 to change, 0 to destroy.`

Read the plan. Names should start with `DH`. On a first run nothing should be changed or destroyed.

Step 8. Apply (this changes Genesys)

```
./scripts/bash/04-apply.sh minimal      # or: full
```

It asks you to type `YES`.

You should see `Apply complete! Resources: N added.`

If it fails partway: read the **last** error, fix the code, run Steps 7 and 8 again. Terraform adds only what is missing.

Step 9. Inventory

```
./scripts/bash/05-inventory.sh
```

You should see counts of objects, then a list of queues beginning `DH`.

Now open Genesys Cloud and look at what was built (`BUILD-MANUAL.md`, "Look at what you built"), then run the tests in the Manual, Part 6.

Step 9b. Health check (read-only)

```
./scripts/bash/08-health-check.sh
```

You should see five numbered checks and `OK  All checks passed`. If a flow is saved but not published, or an expected queue is missing, it prints `FAIL` and exits with 1, so a scheduler or pipeline can alert on it. A `WARN` on the surge switch means someone left it on.

It was tested against a mock server only. If a field name differs in your org, fix the script and note it in your run log.

Step 10. The surge drill

Edit `terraform/envs/full.tfvars`. Remove the `#` from `surge_mode_enabled = true`. Save. Then:

```
./scripts/bash/03-plan.sh full
```

You should see one change: the emergency group `enabled`. Then:

```
./scripts/bash/04-apply.sh full
```

Put the `#` back, plan, apply, to switch it off.

Step 11. Optional: read the org back as Terraform

```
./scripts/bash/06-export.sh
```

You should see `OK Exported to .../export/generated`. Look, but do not commit: it contains real IDs. `export/` is already in `.gitignore`.

Step 12. Tear down

```
./scripts/bash/05-inventory.sh
./scripts/bash/07-destroy.sh
./scripts/bash/05-inventory.sh
```

The destroy script shows what it will delete. Type `DESTROY` to continue. **You should see** `OK Destroyed.` and, in the last inventory, no `DH` queues.

Then check **Admin, Billing** in Genesys and your contract for numbers, add-ons and usage that remain.

Step 13. Put it on GitHub

Run the safety check first:

```
pip install pyyaml python-hcl2
python3 tools/check_repo.py
```

You should see `All checks passed.`

```
git init -b main
git add .
git status
```

Read the list. **You must not see** `.tfstate`, `.tfplan`, `.env`, or `backend.tf`. If you do, stop and fix `.gitignore`.

```
git commit -m "Initial commit: Health-style lab"
git remote add origin https://github.com/asvinluan/genesys-health-style-lab.git
git push -u origin main
```

You should see your files on GitHub and, within a minute or two, the **Actions** tab running the checks.

Run the same checks CI runs

```
        shellcheck -x scripts/bash/*.sh
        python3 tools/check_repo.py
        ( cd terraform/function-actions/normalise-reference && node --test )
        ( cd terraform && tofu fmt -check -recursive && tofu init -backend=false && tofu valid
ate && tofu test )
```

Common errors

What you see	Meaning	Fix
`No module named hcl2`	The checker's two helpers are missing	`pip install pyyaml python-hcl2`
`Permission denied` running a script	Not executable	`chmod +x scripts/bash/*.sh`
`Set GENESYSCLOUD_OAUTHCLIENT_ID`	Variables not set in this shell	Step 3
`Could not get a token`	Wrong ID, secret or region	Re-check Part 2 of the Manual
`Error: 403` or `missing permission`	The OAuth client's role is too narrow	Add the permission named in the error
License or "feature not enabled"	Org lacks AI, WFM, quality or knowledge	Use `minimal`, or set `enable_ai = false` etc.
`already exists`	A previous run left an object	Delete it in Genesys, or import it
Plan wants to destroy something	Code and org disagree	Do not apply. Find out why first.
`Files are not formatted`	Formatting only	`tofu fmt -recursive` in `terraform`

What each script does

Script	Purpose	Changes Genesys
`00-preflight.sh`	Tools, token, org name	No
`01-package-function.sh`	Test and zip the function	No
`02-init.sh`	Format check, init, validate	No
`03-plan.sh`	Show what would be built	No
`04-apply.sh`	Build from the saved plan	**Yes**
`05-inventory.sh`	Count lab objects	No
`06-export.sh`	Read the org back as Terraform	No
`07-destroy.sh`	Delete what this repo built	**Yes**
`08-health-check.sh`	Read-only check that what was built is still healthy	No
`lib.sh`	Shared helpers, sourced not run	No

All scripts stop at the first failure (`set -euo pipefail`). They never print the secret or the token.